Exercise sheet 5[1]

Recall the `Point3D` class described on Exercise Sheet 4. Mathematically, this object is, of course, just a representation of a three-dimensional vector and it would be nice to be able to replicate (some of) the mathematical operations that one can perform with vectors. Furthermore, much of the machinery implemented in 3D actually applies to vectors in any dimension so it makes sense to generalise the class as follows:

```
//=========================================================
/// Class for points in n dimensions
//=========================================================
class Point
{

public:

 /// Default constructor: Specify the spatial dimension and set
 /// the coordinates to zero
 Point(const unsigned& n)
  {
   Coordinates.resize(n,0.0);
  }

 /// Constructor: Pass coordinates as vector
 Point(const std::vector<double>& coord) : Coordinates(coord)
  {}

 /// Function to return the Euclidian distance
 /// to the origin. Const because it doesn't
 /// change the member data
 double distance_to_origin() const
 {
  // How many dimensions to we live in?
  unsigned n=Coordinates.size();

  // n-dimensional Pythagoras
  double tmp=0.0;
  for (unsigned i=0;i<n;i++)
   {
    tmp+=Coordinates[i]*Coordinates[i];
   }
  return std::sqrt(tmp);
 }

 /// Get i-th coordinate (read/write)
 double& operator[](const unsigned& i)
  {
   return Coordinates[i];
  }
```

[1] Version 2026/09/28.
Feedback, comments, corrections, applause, etc. to: `M.Heil@maths.manchester.ac.uk`

2026/09/28, 08:00:00

```
42
43
44 private:
45
46  /// Coordinates stored as vector
47  std::vector<double> Coordinates;
48
49 };
```

Coding Exercises

Q5.1 With the above definition of the `Point` class, vector addition ($\mathbf{c} = \mathbf{a} + \mathbf{b}$) can be done by adding the components of **a** and **b**:

```
1   // Make a point
2   std::vector<double> a_vect={1.0,2.0,3.0};
3   Point a(a_vect);
4
5   // Make another point
6   std::vector<double> b_vect={2.0,4.0,6.0};
7   Point b(b_vect);
8
9   // Make a 3D vector located at the origin
10  unsigned dim=3;
11  Point c(dim);
12
13  // Add a and b manually
14  c[0]=a[0]+b[0];
15  c[1]=a[1]+b[1];
16  c[2]=a[2]+b[2];
```

and we can add **b** to **a** using

```
1
2   // Add b to a manually
3   a[0]=a[0]+b[0];
4   a[1]=a[1]+b[1];
5   a[2]=a[2]+b[2];
```

However, this is all a bit clunky, so modify the `Point` class by overloading the `+=` operator, so that we can do

```
1
2   // Add b to a using an overloaded version of the "+=" operator
3   a+=b;
```

When implementing this, define a macro `PARANOID` so that, if it has been `#defined`, the `+=` operator checks that the two vectors have the same size. (**Hint:** To do this you will have to define at least one additional public member function in the `Point` class.) If the sizes differ, throw a `runtime_error` with a suitable error message.

2026/09/28, 08:00:00

Q5.2 Here's a reminder about the two different ways to create objects and to call their member functions:

You can either create the object (like any other type) and then call the member function via the dot operator. The `[]` operator just attaches itself to the object.

```
1
2   // Make a three-dimensional Point
3   unsigned n=3;
4   Point c(n);
5
6   // Call a member function
7   std::cout << "Distance of point c from origin: "
8             << c.distance_to_origin() << std::endl;
9
10  // Output the x coordinate
11  std::cout << "x-coordinate of point c: " << c[0] << std::endl;
```

Alternatively, you can create the object directly, using `new`, and store a pointer to it. Member functions are then accessible via the `->`:

```
1
2   // Make a Point via a pointer
3   Point* c_pt = new Point(n);
4
5   // Call a member function
6   std::cout << "Distance of point c from origin: "
7             << c_pt->distance_to_origin() << std::endl;
```

However, given a pointer to a `Point` object, how do you output its x-coordinate via the `[]` operator?

Q5.3 Now (re)create the class `Point3D`, to be derived from the `Point` class, such that the point can be constructed in the old form (as on the previous exercise sheet) where the constructor took three doubles:

```
1   // Make a 3D point
2   Point3D a(3.0, 4.0, 1.0);
3
4   // Make another 3D point
5   Point3D b(12.0, 5.0, 1.0);
```

Hint: Inside a class member function, the variable `this` provides a pointer to the class itself.

Q5.4 Add a function

```
1   /// Output coordinates
2   void output() const
```

to the public part of the `Point3D` class and implement it. The function should obtain the three coordinates of the point via the `[]` operator and write them to `std::cout`.

2026/09/28, 08:00:00

Note that the `output` function is (quite rightly!) declared to be const since we wouldn't expect an output function to change the class's member data.

When you try to implement this function you will encounter a problem. Identify the root cause of the problem and make suitable changes to the `Point` base class – but note that these changes **must not** change the way in which the coordinates themselves are stored. They must remain in the private vector `Coordinates` in the `Point` base class!

Q5.5 Throwing and catching errors to create a better Newton's method

Remember that Newton's method is an algorithm for finding the roots x of a non-linear equation, $f(x)$, i.e. the values of x for which $f(x) = 0$. Starting from some initial guess for the location of the root, x_0, the Newton method produces the iterates

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)},$$

for $n = 0, 1, 2, \ldots$. If the initial guess is close enough to a root of f, then the sequence x_n converges to it.

Providing a "good" initial guess that ensures the convergence of the iteration is often difficult. However, there are many problems that contain a parameter and one may be interested in finding the roots for a whole range of these parameters. Assuming that the root is "easy" to find for a certain value of the parameter, one can then use so-called continuation methods to compute the the roots for others.

To illustrate the idea we consider the function

$$f(x; A) = x^{1/4} - 101 + A,$$

whose only (real) root is given by

$$x(A) = (101 - A)^4.$$

Note that we have explicitly indicated the dependence of the root on the parameter A.

For $A = 1$, the solution is particularly simple, $x(A = 1) = 10^8$. Assume now that we don't know the exact solution for any other values of A but wish to compute x for $A = 100$. Using the initial guess $x_0 = 10^8$ (i.e. the solution for $A = 1$) is not good enough – the Newton method diverges (try it!).

However, the solution for $A = 1$ is likely to be a good initial guess for the solution at $A = 2$, say. This suggests the following strategy:

(a) Choose $A_{\min} = 1$ and $A_{\max} = 100$ and decide on a (small) increment $\Delta A \ll A_{\max} - A_{\min}$.

(b) Use $x_{\text{init}} = x(A_{\min})$ (the known solution) as an initial guess for $x(A_{\min} + \Delta A)$.

(c) Call Newton's method, which will hopefully converge to yield $x(A_{\min} + \Delta A)$.

(d) Now use this result as an initial guess for the Newton iteration for $x(A_{\min} + 2\Delta A)$.

2026/09/28, 08:00:00

(e) Keep going until we reach $A_{\max}$.

Implement the above method (feel free to recycle code you've written for Analysis Exercise 2), and explore how large you can make the increment ΔA without causing the Newton method to diverge as you approach $A = 100$.

My own experiments suggest that for $\Delta A > 5$, the Newton method fails to converge at some point as A approaches $A_{\max}$, implying that this increment is too big. We can now either restart the entire computation with a smaller value (thus performing many more Newton solves, even in a region where the method worked fine with a larger increment – wasteful!), or be more clever about it. Rewrite your code so that the failing Newton iteration throws a runtime error. You can then catch this error and restart the iteration from the most recently converged solution, but with a smaller increment. Do it!

Q5.6 The continuation approach is particularly powerful in in cases where the function is nonlinear but the size of the nonlinearity is controlled by a parameter. For instance the function

$$f(x) = A^2 \exp(x^2 \sin(20x)) + x$$

has a unique (and obvious!) root at $x = 0$ when $A = 0$. This root therefore provides a sensible initial guess for the solution at small but finite values of A, and we can then continue away from this value. How can this idea help us with functions that don't have a parameter, such as

$$f(x) = \exp(x^2 \cos(20x)) + x$$

2026/09/28, 08:00:00